НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №6. Робота зі складними формами та валідація (Actions React 19, React Hook Form, Zod)

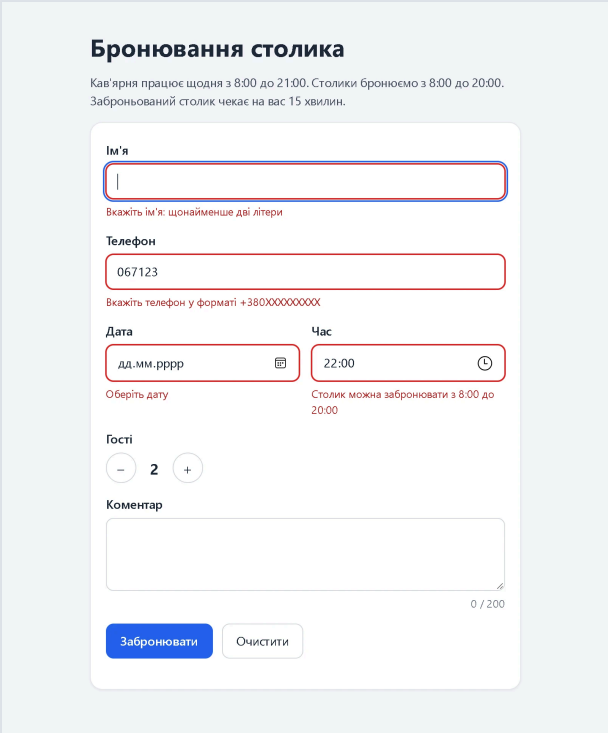
з дисципліни «Розробка вебзастосунків на React»

> **Примітка щодо актуальності.** Документ підготовлено станом на 12 листопада 2026 року: React 19, React Hook Form 7, Zod 4, `@hookform/resolvers` 7, Vite 8, ESLint 10 з `eslint-plugin-react-hooks` 7, Prettier 3, React DevTools 8. У разі відмінностей орієнтуйтеся на документацію React і бібліотек (§2).

1. Мета роботи

Навчитися будувати складні форми в React: описати правила валідації однією схемою Zod, яку використовують і форма, і сервер, передати значення форми React Hook Form, показувати доступні повідомлення про помилки й надсилати дані через Action React 19 зі станом очікування та відповіддю сервера.

Очікуваний результат: у теці `lab-6/` репозиторію `react-labs` — форма бронювання, яка після надсилання з помилками показує повідомлення під кожним неправильним полем, усі одразу, і ставить курсор у перше з них; повідомлення з'являються, коли користувач залишає поле, і зникають, щойно значення виправлено; сервер відповідає за секунду, а поки чекає, кнопка показує «Надсилаємо…»; на 19:00 сервер відмовляє, і значення лишаються в полях; після успішного бронювання форма очищається, а підтвердження лишається; «Очистити» очищає все, разом із відповіддю сервера; застосунок відкривається на Vercel. За додаткові бали — форма підписки на новини без бібліотек форм (на занятті) і та сама форма бронювання з іншими бібліотеками (удома).



2. Корисні посилання

Теоретичної частини в цій роботі немає. Перед заняттям прочитайте сторінки за посиланнями 1 і 2. Решту відкривайте, коли до них відсилає завдання. Українською є лише сторінка 1, решта — англійською. Для §4.8 знадобиться ще документація бібліотек, які ви оберете.

1. Реагування станом на введення — стани форми (введення, надсилання, успіх, помилка) і як перемикати їх станом React.
2. React Hook Form: Get Started — реєстрація полів, правила, контрольовані компоненти, валідація схемою, помилки (§4.3–4.4).
3. Довідка React Hook Form: useForm, register, Controller, useWatch, reset, setFocus (§4.3–4.5).
4. Zod: Basics, Defining schemas, Customizing errors — схема, перевірка даних і власні повідомлення про помилки (§4.2).
5. @hookform/resolvers — адаптери між бібліотеками схем і React Hook Form, серед них `zodResolver` (§4.4).
6. `useActionState` і `startTransition` — Actions React 19: результат дії, стан очікування, черга викликів (§4.5).
7. W3C WAI: Validating Input і User Notifications — як перевіряти введення й доступно повідомляти про помилки. На них спирається самостійна робота «Проєктування доступних форм».
8. MDN: `aria-invalid`, `aria-describedby`, `noValidate` — атрибути з §4.4.
9. `<form>` і `useFormStatus` — форма з атрибутом `action` і стан її надсилання (§4.7).

3. Завдання лабораторної роботи

1. Додати в репозиторій `react-labs` теку `lab-6/` зі стартового проєкту й переконатися, що форма приймає неправильні дані (§4.1).
2. Описати правила форми схемою Zod (§4.2).
3. Передати значення форми React Hook Form (§4.3).
4. Показувати повідомлення про помилки під полями (§4.4).
5. Надсилати бронювання на сервер через Action (§4.5).
6. Перевірки, GitHub і Vercel (§4.6), звіт (§5).
7. **+1 бал:** форма підписки без бібліотек форм: `<form action>`, `useFormStatus` (§4.7).
8. **+1 бал, домашнє завдання:** та сама форма бронювання з іншими бібліотеками (§4.8).

§4.1–4.6 — основна частина, вона оцінюється в 1 бал; усі її завдання виконуються на занятті (орієнтовно 20–30 хвилин). §4.7 дає ще 1 бал; беріться за неї на занятті, лише коли §4.6 відправлено на GitHub. §4.8 — домашнє завдання ще на 1 бал; його коміти мають бути на GitHub до захисту ЛР-6. Звіт (§5) можна надіслати після заняття.

AI. Код цієї роботи, зокрема §4.8, пишіть самостійно. AI-автодоповнення в редакторі можна залишити ввімкненим, але кожен запропонований рядок ви маєте вміти пояснити на захисті. Доручати роботу AI-агентові не рекомендовано: мета — навчитися самому вирішувати, де живе кожне правило й кожне значення форми.

4. Хід виконання роботи

Форма та сама, що наприкінці ЛР-5. Що в ній нового, перелічено в `lab-6/README.md`: поле «Телефон», порожні абзаци для повідомлень про помилки, імітований сервер. У ЛР-5 значення полів читав `FormData`, а перевіряв їх лише браузер (наприклад, `required`). Тепер правила переїдуть у схему, значення — у бібліотеку форм, а дані підуть на сервер. Після кожного кроку перевіряйте сторінку в браузері, консоль і вкладку **Components** ⚛. Файли зберігайте як у ЛР-1 (`Ctrl+S`): Prettier відформатує їх під час збереження.

4.1. Стартовий проєкт

1. Відкрийте в редакторі теку `react-labs` і перевірте, що `git status` чистий. Якщо працюєте на іншому комп'ютері, спершу клонуйте `react-labs` з GitHub.
2. Завантажте lab-6.zip і розпакуйте його в провіднику (Windows: **Extract All**, macOS: подвійний клік). Вийде тека `lab-6`. Перемістіть її в корінь `react-labs`, поруч із теками попередніх робіт.
   Не розпаковуйте архів командою в терміналі в корені `react-labs`: файли в ньому лежать без теки й замінили б кореневі `package.json` і `.gitignore`.

3. У корені репозиторію виконайте `git status -u`: усі нові файли мають бути в `lab-6/`.
4. Встановіть залежності в корені й запустіть dev-сервер:

   ```
   npm install
   cd lab-6
   npm run dev
   ```

   `npm install` встановить і три нові бібліотеки: `react-hook-form`, zod і `@hookform/resolvers`. Відкрийте http://localhost:5173: на сторінці форма бронювання з ЛР-5 з новим полем «Телефон». Dev-сервер не зупиняйте до §4.6, а для команд git відкрийте другий термінал.
5. Спробуйте форму, поки в ній немає правил:
   1. Натисніть «Забронювати» з порожніми полями: браузер попросить заповнити ім'я, і лише його. Про кожне наступне порожнє поле він скаже окремо, після наступного натискання.
   2. Введіть ім'я «О», телефон «123», учорашню дату й час 03:00 і натисніть «Забронювати»: форма все прийняла, і підтвердження показує ці дані. Над формою написано «Столики бронюємо з 8:00 до 20:00», але цього ніхто не перевіряє.
6. Що вже є в проєкті:

Файл	Що в ньому
`src/App.jsx`	заголовок, опис кав'ярні й `BookingForm`
`src/components/BookingForm.jsx`	форма з ЛР-5 після §4.6, поле «Телефон» і порожні абзаци `…-error` для повідомлень про помилки
`src/components/GuestsStepper.jsx`	контрольований компонент із ЛР-5: пропси `value` і `onChange`
`src/bookingSchema.js`	функції `today()` і `nowTime()` і порожня схема `bookingSchema`, яку ви заповните в §4.2
`src/api.js`	імітований сервер: `sendBooking(booking)` для §4.5 і `subscribe(email)` для §4.7
`src/describeBooking.js`	функція з ЛР-5, яка складає текст підтвердження
`src/index.css`	усі стилі сторінки, зокрема для повідомлень про помилки, стану очікування й форми підписки (§4.7)
`package.json`, `eslint.config.js`, `.prettierrc.json`	скрипти й налаштування як у ЛР-1; у `dependencies` — три нові бібліотеки

7. Зробіть коміт у корені репозиторію: так до нього потрапить і `package-lock.json`.

   ```
   git add .
   git commit -m "chore: add lab 6 starter"
   ```

Далі шляхи до файлів (`src/components/BookingForm.jsx`) — відносно `lab-6/`.

4.2. Правила форми в схемі Zod

Схема описує, якими мають бути дані: кожне поле, його тип і правила. Zod перевіряє за схемою будь-який об'єкт і повертає або дані, або список помилок. Схема — звичайний JavaScript, тож перевірити її можна ще до того, як нею скористається форма.

У `src/bookingSchema.js` замініть рядок `export const bookingSchema = z.object({})` на схему:

```
export const bookingSchema = z
  .object({
    name: z.string().trim().min(2, "Вкажіть ім'я: щонайменше дві літери"),
    phone: z
      .string()
      .trim()
      .regex(/^\+380\d{9}$/, 'Вкажіть телефон у форматі +380XXXXXXXXX'),
    date: z
      .string()
      .min(1, 'Оберіть дату')
      .refine((date) => date >= today(), 'Ця дата вже минула'),
    time: z
      .string()
      .min(1, 'Оберіть час')
      .refine(
        (time) => time >= '08:00' && time <= '20:00',
        'Столик можна забронювати з 8:00 до 20:00',
      ),
    guests: z.coerce
      .number()
      .int()
      .min(1, 'Щонайменше 1 гість')
      .max(8, 'Не більше 8 гостей'),
    comment: z.string().trim().max(200, 'Коментар — до 200 символів'),
  })
  .refine((data) => data.date !== today() || data.time > nowTime(), {
    error: 'Цей час сьогодні вже минув',
    path: ['time'],
  })
```

- `z.object({ … })` — схема об'єкта: для кожного поля форми своя схема;
- `z.string()` — рядок. `trim()` прибирає пробіли на початку й у кінці, перш ніж перевіряти решту правил; `min(2, '…')` і `max(200, '…')` — найменша й найбільша довжина, а другий аргумент — повідомлення про помилку;
- `regex(…)` — рядок має відповідати регулярному виразу: `+380` і рівно дев'ять цифр;

- `refine(функція, '…')` — правило, для якого в Zod немає готової перевірки: функція отримує значення й повертає `true`, якщо воно правильне. Дата й час у форматах полів (`2026-11-14`, `18:30`) порівнюються як рядки;
- `z.coerce.number()` спершу перетворює значення на число: сервер і `FormData` отримують рядки, і `'3'` має стати `3`. Далі `int()`, `min()` і `max()` перевіряють уже число;
- `refine` на всьому об'єкті перевіряє два поля разом: якщо дата сьогоднішня, час має бути пізнішим за поточний. `path: ['time']` прив'язує цю помилку до поля «Час», а `error` задає її повідомлення.

Перевірка в консолі. Схему можна перевірити ще без форми. На сторінці з dev-сервером відкрийте DevTools → **Console** і введіть:

```
const { bookingSchema } = await import('/src/bookingSchema.js')
```

`import()` завантажує модуль так само, як `import` на початку файлу, а dev-сервер Vite віддає файли з `src/` за їхніми шляхами. Першого разу Vite може перезавантажити сторінку: він щойно знайшов бібліотеку `zod`, яку ще не імпортує жоден компонент, і підготував її. Тоді введіть рядок ще раз.

Перевірте неправильні дані, а потім правильні. Рядки об'єкта переносьте клавішами `Shift+Enter`, а виконуйте вираз клавішею `Enter`:

```
bookingSchema.safeParse({
  name: 'O',
  phone: '067',
  date: '2020-01-01',
  time: '22:00',
  guests: 9,
  comment: '',
})
```

```
bookingSchema.safeParse({
  name: '  Олена ',
  phone: '+380671234567',
  date: '2030-01-01',
  time: '18:00',
  guests: '3',
  comment: '',
})
```

Що має вийти:

- перший `safeParse` повертає `success: false`; розгорніть `error`: у масиві `issues` п'ять помилок, у кожної `path` (поле) і `message` (ваше повідомлення);
- другий повертає `success: true` і `data`, у якому ім'я без пробілів, а `guests` — число `3`;

- сторінка поводиться як у §4.1: схемою поки ніхто не користується.

Коміт: `feat: describe the booking rules with a Zod schema`. **Довідка:** Basics, Strings, Numbers, Coercion, Refinements, Customizing errors.

4.3. React Hook Form бере значення форми

У ЛР-5 значення полів зберігали браузер і стан `BookingForm` (коментар і гості), а очищення ви написали самі. Тепер значення переходять до бібліотеки React Hook Form. Правил на цьому кроці ще немає: сторінка має поводитися точнісінько як наприкінці ЛР-5, це й перевірка.

1. У `src/components/BookingForm.jsx` замініть імпорт з `react`:

   ```
   import { useState } from 'react'
   import { Controller, useForm, useWatch } from 'react-hook-form'
   ```

   Над функцією `BookingForm` оголосіть початкові значення всіх полів:

   ```
   const defaultValues = {
     name: '',
     phone: '',
     date: '',
     time: '',
     guests: 2,
     comment: '',
   }
   ```

   У самому компоненті залиште стан `booking`, а стани `comment` і `guests` і реф `nameRef` замініть формою:

   ```
   const { register, control, handleSubmit, reset, setFocus } = useForm({
     defaultValues,
   })
   ```

   `useForm` створює форму й повертає функції для роботи з нею. До `defaultValues` форма повертається після `reset()`.
2. **`register`.** У полях «Ім'я», «Телефон», «Дата», «Час» і «Коментар» замініть атрибут `name="…"` на `{...register('…')}` з тією самою назвою. У полі «Ім'я» приберіть ще й `ref={nameRef}`, а в коментарі — `value` і `onChange`:

   ```
   <input {...register('name')} autoComplete="name" required />
   ```

   ```
   <textarea
     {...register('comment')}
     maxLength={200}
     aria-describedby="comment-counter"
   />
   ```

`register('name')` повертає об'єкт із пропсами `name`, `onChange`, `onBlur` і `ref`, а `{...}` передає їх полю. Значення лишається в DOM, як було з іменем у ЛР-5: React Hook Form читає його через реф, коли потрібно.

3. **`Controller`.** Замініть `<GuestsStepper value={guests} … />` на:

```
<Controller
  name="guests"
  control={control}
  render={({ field }) => (
    <GuestsStepper value={field.value} onChange={field.onChange} />
  )}
/>
```

`GuestsStepper` — контрольований компонент (ЛР-5, §4.6): власного стану в нього немає, він чекає на `value` і `onChange`. DOM-поля, якому `register` міг би передати реф, у нього теж немає. `Controller` зберігає значення у формі й передає компоненту в `field` саме `value` і `onChange`. У `GuestsStepper.jsx` видаліть приховане поле `<input type="hidden" name="guests" value={value} />`: `FormData` більше ніхто не читає.

4. **Лічильник.** Під `useForm` додайте:

```
const comment = useWatch({ control, name: 'comment' })
```

Рядок `{comment.length} / 200` лишається як був. `useWatch` підписує `BookingForm` на одне поле: компонент рендериться, коли змінюється коментар. У прикладах документації часто трапляється `watch('comment')` з `useForm`, але ESLint позначає його попередженням `react-hooks/incompatible-library` («React Hook Form's `useForm()` API returns a `watch()` function which cannot be memoized safely»), тож у цій роботі — `useWatch`.

5. **Надсилання.** Видаліть свою функцію `handleSubmit` з ЛР-5, а замість неї додайте:

```
function onValid(data) {
  setBooking(data)
}
```

Формі передайте `onSubmit={handleSubmit(onValid)}`. Тепер `handleSubmit` — функція React Hook Form: вона скасовує стандартну поведінку, збирає значення всіх полів, перевіряє їх і, якщо все гаразд, викликає `onValid` з об'єктом даних. У JSX її викликають, і це правильно: `handleSubmit(onValid)` повертає обробник події. `handleKeyDown` з ЛР-5 не змінюється: `requestSubmit()` запускає подію `submit`, яку тепер обробляє `handleSubmit`.

6. **Очищення.** Замініть `handleReset`:

```
function handleReset() {
  setFocus('name') // до reset(): він прибирає посилання на поля
  reset()
  setBooking(null)
}
```

`reset()` повертає поля до `defaultValues`, а `setFocus('name')` ставить курсор у поле «Ім'я» через реф, який полю передав `register`. Порядок важливий: `reset()` прибирає посилання на поля до наступного рендеру, і `setFocus` після нього нічого б не зробив.
«Очистити» стає звичайною кнопкою з обробником, а `onReset` з форми видаліть:

```
<button type="button" onClick={handleReset}>
  Очистити
</button>
```

Причина: `reset()` React Hook Form скидає й саму форму в браузері (`form.reset()`), а отже запускає подію `reset`. З `onReset` на формі `handleReset` спрацьовував би щоразу, коли форму скидає код, а в §4.5 код скидатиме її після кожного успішного бронювання.

Що має вийти:

- сторінка працює так само, як наприкінці ЛР-5: підтвердження після «Забронювати», лічильник, кнопки − і +, `Ctrl+Enter`, «Очистити» з курсором у полі «Ім'я»;
- з порожнім ім'ям браузер, як і раніше, просить його заповнити: перевіряє поки лише `required`;
- у **Components** ❁ між `BookingForm` і `GuestsStepper` з'явився `Controller`, а пропси `GuestsStepper` ті самі: `value` і `onChange`; серед хуків `BookingForm` немає станів коментаря й гостей і рефа: значення тепер зберігає React Hook Form;
- ESLint (у редакторі чи `npm run lint` у другому терміналі) не показує ні помилок, ні попереджень.

Коміт: `refactor: keep the form values in React Hook Form` (`refactor`, бо сторінка не змінилася). **Довідка:** Register fields, Integrating Controlled Inputs, useWatch, reset, setFocus.

4.4. Повідомлення про помилки

Тепер форма перевірятиме значення схемою з §4.2 і показуватиме повідомлення під полями.

1. **Схема у формі.** Імпортуйте адаптер і схему:

```
import { zodResolver } from '@hookform/resolvers/zod'
import { bookingSchema } from '../bookingSchema.js'
```

   і передайте їх `useForm`, а з результату візьміть ще й помилки:

```
const {
  register,
  control,
  handleSubmit,
  reset,
  setFocus,
  formState: { errors },
} = useForm({
  resolver: zodResolver(bookingSchema),
  defaultValues,
  mode: 'onTouched',
})
```

   - `zodResolver(bookingSchema)` — адаптер із пакета `@hookform/resolvers`: React Hook Form перевіряє значення схемою Zod і кладе її помилки в `errors`, по одній на поле. `errors.name.message` — перше повідомлення для імені;
   - `mode` — коли перевіряти поле: `'onSubmit'` (типово) — лише під час надсилання; `'onBlur'` — коли поле залишають; `'onChange'` — з кожною зміною, навіть поки людина ще вперше вводить значення; `'onTouched'` — уперше, коли поле залишають, а далі з кожною зміною. Останній не показує помилок, поки людина не закінчила вводити, але одразу показує, що помилку виправлено.

2. **`noValidate`.** Додайте атрибут формі:

```
<form
  className="booking-form"
  noValidate
  onSubmit={handleSubmit(onValid)}
  onKeyDown={handleKeyDown}
>
```

   `noValidate` вимикає перевірку браузера: без нього браузер показував би свої підказки й не пропускав форму до `handleSubmit`. Атрибути `required` на полях лишаються: за ними програми зчитування з екрана оголошують поле обов'язковим.

3. **Повідомлення.** Поле «Ім'я» з його абзацом має стати таким:

```
<div className="field">
  <label>
    Ім'я
    <input
      {...register('name')}
      autoComplete="name"
      required
      aria-invalid={errors.name ? 'true' : 'false'}
      aria-describedby="name-error"
    />
  </label>
  <p id="name-error" className="field-error">
    {errors.name?.message}
  </p>
</div>
```

   - `errors.name?.message` — повідомлення зі схеми або undefined, якщо помилки немає. Порожній абзац стилі ховають;
   - `aria-invalid` повідомляє програмі зчитування з екрана, що значення поля неправильне, а в стилях дає червону рамку;
   - `aria-describedby` пов'язує поле з повідомленням: на полі програма зчитування прочитає і підпис, і текст помилки;
   - абзац стоїть поза `<label>`: інакше текст помилки став би частиною назви поля, а `<p>` усередині `<label>` — помилка в HTML.

   Так само зробіть поля «Телефон» (phone), «Дата» (`date`) і «Час» (`time`) з їхніми абзацами `phone-error`, `date-error` і `time-error`.
4. **Чому не HTML?** Майже кожне правило схеми браузер уміє перевірити сам: `required`, `minLength`, `pattern` для телефону, `min` і `max` для дати й часу. Бібліотеки тут потрібні для іншого: повідомлення українською біля кожного поля, а не підказка браузера; усі помилки одразу, а не по одній; вибір моменту перевірки (onTouched); одна схема і для форми, і для сервера; правило для двох полів, яке атрибутами не описати; значення з власного компонента (`GuestsStepper`).
5. **Правило для двох полів.** Виберіть сьогоднішню дату й час, який уже минув (наприклад, 08:00), і перейдіть до іншого поля: під часом з'явиться «Цей час сьогодні вже минув». Тепер змініть дату на завтрашню: повідомлення лишилося, хоча час уже правильний. Після зміни поля React Hook Form оновлює помилки лише цього поля, а про час дата не знає. Додайте в `register` дати залежність:

```
{...register('date', { deps: 'time' })}
```

   deps просить після кожної зміни дати перевірити й поле «Час». Повторіть: тепер повідомлення зникає, щойно змінюється дата.

Що має вийти:

- «Забронювати» з порожньою формою показує чотири повідомлення одразу, рамки цих полів червоні, а курсор стоїть у полі «Ім'я»: React Hook Form сам переводить фокус на перше неправильне поле;
- поки ви вперше вводите значення, повідомлення немає; воно з'являється, коли ви залишаєте поле, і зникає, щойно значення виправлено;
- час 22:00 дає «Столик можна забронювати з 8:00 до 20:00»;
- з `deps` повідомлення про час, що минув, зникає після зміни дати;
- якщо ввести ім'я з пробілами на початку й у кінці, у підтвердження воно потрапляє без них: `onValid` отримує дані, які повернула схема;
- «Очистити» прибирає й повідомлення про помилки.

Коміт: `feat: show validation errors next to the fields`. **Довідка:** Schema Validation, Handle errors, useForm: mode, @hookform/resolvers: Zod, Validating Input, `aria-invalid`, `aria-describedby`.

4.5. Надсилання через Action

Справжня форма надсилає дані на сервер і чекає на відповідь. У `src/api.js` уже є імітований сервер: `sendBooking(booking)` відповідає за секунду, перевіряє дані **тією самою схемою** (`bookingSchema.safeParse`) і відмовляє, якщо на обраний час столиків немає. Перевірка на клієнті потрібна людині, яка заповнює форму; перевірка на сервері — гарантія, бо надіслати запит можна й без вашої форми. А про вільні столики знає лише сервер.

Асинхронну функцію, яку React виконує в транзиції (Transition), React 19 називає **дією** (Action). Хук `useActionState` зберігає результат останньої дії й повідомляє, чи вона ще виконується.

1. **Дія.** Замініть імпорт з `react` і `describeBooking` на:

   ```
   import { startTransition, useActionState } from 'react'
   ```

   ```
   import { sendBooking } from '../api.js'
   ```

   Видаліть стан `booking`: відповідь сервера зберігатиме `useActionState`, а текст підтвердження складає сервер. Під `useWatch` додайте:

```
const [result, dispatch, isPending] = useActionState(bookTable, null)

// Дія для useActionState: null — сигнал «Очистити»; інакше надсилаємо
// бронювання, після успіху очищаємо форму й повертаємо відповідь.
async function bookTable(previousResult, booking) {
  if (booking === null) return null
  const response = await sendBooking(booking)
  if (response.ok) startTransition(() => reset())
  return response
}
```

- `useActionState(дія, початковий стан)` повертає три значення: результат останнього виклику дії (`null`, поки викликів не було), функцію, що запускає дію, і `isPending` — `true`, поки дія виконується;
- дія отримує попередній результат і те, що передали в `dispatch`; що вона поверне, те й стане новим `result`. Відповідь сервера має вигляд `{ ok, message }`;
- `bookTable` оголошено всередині компонента, бо їй потрібна `reset` з `useForm`;
- `reset()` після `await` знову загорнуто в `startTransition`: оновлення стану після `await` уже не належить до транзиції дії;
- `null` замість бронювання — домовленість цієї форми, так «Очистити» скидатиме відповідь сервера (крок 4). Окремої функції для скидання `useActionState` не має.

2. **Запуск.** Замініть `onValid`:

```
function onValid(data) {
  if (isPending) return
  startTransition(() => dispatch(data))
}
```

   - `dispatch` викликають у `startTransition`: так React знає, що це дія, і показує стан очікування. Форма й далі має `onSubmit={handleSubmit(onValid)}`, а не `action`: спершу дані перевіряє React Hook Form, і в дію йдуть лише правильні, тож транзицію ви запускаєте самі;
   - `useActionState` ставить виклики в чергу й виконує їх по одному. Без перевірки `isPending` повторне натискання «Забронювати» надіслало б ще одне бронювання.

3. **Очікування й відповідь.** Кнопка надсилання та абзац для відповіді:

```
<button type="submit" aria-disabled={isPending}>
  {isPending ? 'Надсилаємо…' : 'Забронювати'}
</button>
```

```
<p
  role="status"
  className={result?.ok === false ? 'status error' : 'status'}
>
  {result?.message}
</p>
```

Кнопка отримує `aria-disabled`, а не `disabled`, з тієї самої причини, що й кнопки гостей у ЛР-5: кнопка, яка стає `disabled`, втрачає фокус. Натискання під час очікування відкидає перевірка `isPending` в `onValid`. Клас `error` робить відмову сервера червоною.

4. **Скидання відповіді.** У `handleReset` замініть `setBooking(null)` на:

```
startTransition(() => dispatch(null))
```

5. **Без транзиції.** Тимчасово замініть в `onValid` `startTransition(() => dispatch(data))` на `dispatch(data)` і надішліть форму: кнопка не показує «Надсилаємо…», а в консолі помилка React «An async function with useActionState was called outside of a transition. This is likely not what you intended (for example, isPending will not update correctly)…». Поверніть `startTransition`.

Що має вийти:

- після «Забронювати» кнопка приблизно секунду показує «Надсилаємо…», а в консолі з'являється рядок «Сервер отримав бронювання»;
- на 19:00 сервер відповідає червоним «На 19:00 вільних столиків немає. Оберіть інший час.», а всі значення лишаються в полях;
- на інший час з'являється підтвердження від сервера, а форма очищається: поля порожні, гостей 2, лічильник «0 / 200»;
- натискання «Забронювати», поки видно «Надсилаємо…», не додає в консоль другого рядка «Сервер отримав бронювання»;
- «Очистити» прибирає відповідь сервера й ставить курсор у поле «Ім'я»;
- `Ctrl+Enter` надсилає форму, як і раніше.

Коміт: `feat: send the booking with an action`. **Довідка:** Adding state to an action, How useActionState queuing works, Reset state, Async function outside transition, `startTransition`.

4.6. Перевірки, GitHub і Vercel

1. Зупиніть dev-сервер (`Ctrl+C`) і виконайте в `lab-6/`:

```
npm run lint
npm run format:check
npm run build
```

Усі три команди мають пройти без помилок, а `npm run lint` — ще й без попереджень (`warning`). Якщо `format:check` назве файл, відкрийте його, збережіть ще раз і закомітьте: `style: format lab 6 files`.

2. Відправте коміти на GitHub: `git push`. Історія комітів основної частини:
    1. `chore: add lab 6 starter`;
    2. `feat: describe the booking rules with a Zod schema`;
    3. `refactor: keep the form values in React Hook Form`;
    4. `feat: show validation errors next to the fields`;
    5. `feat: send the booking with an action`.
3. На vercel.com → **Add New** → **Project** ще раз імпортуйте `react-labs`: це буде ще один проєкт з того самого репозиторію. Перед **Deploy** налаштуйте:

Поле	Значення
Project Name	`react-lab-6`
Root Directory	**Edit** → `lab-6`
Framework Preset	Vite (Vercel розпізнає його сам)
Output Directory	`dist`

4. Після деплою скопіюйте адресу застосунку з вкладки **Overview** проєкту, поле **Domains**, відкрийте її й забронюйте столик ще раз.

На цьому основну частину роботи завершено.

4.7. +1 бал: форма підписки без бібліотек форм

Беріться за це завдання, лише коли §4.6 відправлено на GitHub і застосунок відкривається на Vercel. Запустіть dev-сервер знову (`npm run dev`).

Під формою бронювання з'явиться ще одна, з одним полем: email для новин кав'ярні. Для одного поля бібліотека форм не потрібна. У React 19 функцію можна передати в

атрибут `action` форми: React сам запускає її в транзиції, передає їй `FormData`, а після її завершення очищає неконтрольовані поля форми.

1. Створіть `src/components/SubscribeForm.jsx`:

```
import { useActionState } from 'react'
import { useFormStatus } from 'react-dom'
import { z } from 'zod'
import { subscribe } from '../api.js'

const emailSchema = z.email('Вкажіть адресу на кшталт olena@gmail.com')

function SubmitButton() {
  const { pending } = useFormStatus()
  return (
    <button type="submit" aria-disabled={pending}>
      {pending ? 'Підписуємо…' : 'Підписатися'}
    </button>
  )
}

async function subscribeAction(previousState, formData) {
  const email = formData.get('email')
  const result = emailSchema.safeParse(email)
  if (!result.success) {
    return { ok: false, message: result.error.issues[0].message }
  }
  return subscribe(result.data)
}

export default function SubscribeForm() {
  const [state, formAction] = useActionState(subscribeAction, null)

  return (
    <form className="subscribe-form" action={formAction} noValidate>
      <label>
        Email для новин
        <input
          type="email"
          name="email"
          autoComplete="email"
          aria-invalid={state?.ok === false ? 'true' : 'false'}
          aria-describedby="subscribe-status"
        />
      </label>
      <SubmitButton />
      <p
        id="subscribe-status"
        role="status"
        className={state?.ok === false ? 'status error' : 'status'}
      >
```

```
        {state?.message}
      </p>
    </form>
  )
}
```

- `useActionState` той самий, що в §4.5, але функцію `formAction` отримує атрибут `action`, тож React сам передає дії `FormData` і сам запускає транзицію;
- адресу перевіряє Zod усередині дії: `z.email()` — готова схема для email, а повідомлення береться з першої помилки в `issues`;
- `subscribe` з `api.js` відповідає за секунду; адреси на `@example.com` вважає вже підписаними;
- `SubmitButton` — окремий компонент, бо `useFormStatus` повідомляє стан тієї форми, **всередині** якої компонент рендериться. У самому `SubscribeForm` форма ще не батьківський елемент, і `pending` завжди був би `false`.

2. У `src/App.jsx` імпортуйте `SubscribeForm` і додайте під `<BookingForm />`:

```
<section>
  <h2>Новини кав'ярні</h2>
  <SubscribeForm />
</section>
```

3. Спробуйте три адреси:
   1. неправильну, наприклад `olena@`: з'являється повідомлення, а поле порожнє;
   2. зайняту, `olena@example.com`: секунду кнопка показує «Підписуємо…», потім — «Ця адреса вже підписана.», і поле знову порожнє;
   3. правильну, наприклад `olena@gmail.com`: підтвердження, і поле порожнє, як і має бути після підписки.

   React очищає поле після кожної дії, навіть після відмови. Людині з опечаткою довелося б вводити адресу заново.
4. Нехай неправильна адреса лишається в полі. Поверніть її з дії разом із повідомленням:

```
return { ok: false, message: result.error.issues[0].message, email }
```

   і додайте полю `defaultValue={state?.email}`. Після дії React повертає поле до його початкового значення, `defaultValue`, а тепер це адреса з останньої відповіді. Для зайнятої адреси `subscribe` уже повертає `email`.
5. Прочитайте й порівняйте з формою бронювання. Там `<form action>` не підходить: React очищав би поля після кожної відповіді, навіть після відмови на 19:00, і стирав би значення, які зберігає React Hook Form. Тому форма бронювання надсилається через `onSubmit`, а очищає її `reset()` у дії, лише після успіху. У формі

підписки поле одне, перевірка — один рядок, а автоматичне очищення після підписки — саме те, що треба.

Що має вийти:

- три випадки з кроку 3; після кроку 4 неправильна й зайнята адреси лишаються в полі, а після успішної підписки поле порожнє;
- `npm run lint`, `npm run format:check` і `npm run build` проходять, як у §4.6;
- Vercel показує сторінку з формою підписки.

Коміт: `feat: add the newsletter form with a form action`, відправлений на занятті.
Довідка: Display a pending state during form submission, Handling form submission errors, `useFormStatus`, Zod: emails.

4.8. +1 бал, домашнє завдання: ті самі вимоги, інші інструменти

React Hook Form і Zod — не єдині бібліотеки для форм і схем. Зберіть ту саму форму бронювання з іншими й порівняйте. Беріться за це завдання, коли §4.6 відправлено на GitHub; коміти мають бути на GitHub до захисту ЛР-6. Коду тут немає: бібліотеки ви вибираєте самі й розбираєтеся з їхньою документацією.

1. **Копія.** У провіднику скопіюйте теку `lab-6/` у `lab-6-alt/`, без `node_modules/` і `dist/`. У `lab-6-alt/package.json` змініть `"name"` на `"lab-6-alt"` і лише тоді виконайте `npm install` у корені `react-labs`. Двох пакетів з однаковою назвою npm не прийме, а команди з кроку 2 нічого не змінять, доки `npm install` не додасть нову теку до робочих просторів. Коміт: `chore: copy lab 6 for the alternative version`.
2. **Бібліотеки.** Замініть React Hook Form іншою бібліотекою форм: Formik, TanStack Form, React Final Form чи іншою. Zod можна залишити або теж замінити (Yup, Valibot, Joi...). У корені репозиторію, наприклад:

   ```
   npm uninstall react-hook-form @hookform/resolvers -w lab-6-alt
   npm install formik yup -w lab-6-alt
   ```

   Якщо замінюєте й Zod, а `SubscribeForm` з §4.7 її не використовує, додайте до `npm uninstall` ще `zod`.
3. **Форма.** Перепишіть `BookingForm` (і схему, якщо замінили бібліотеку схем) так, щоб сторінка поводилася як `lab-6` після §4.6: ті самі правила й повідомлення, `aria-invalid` і `aria-describedby`, курсор у першому неправильному полі, «Надсилаємо...», відповідь сервера, очищення після успіху, «Очистити». `api.js` має перевіряти дані схемою цієї версії. `SubscribeForm`, якщо ви її зробили, лишається як є.
4. **Порівняння.** Додайте в `lab-6-alt/README.md` розділ «Порівняння з React Hook Form і Zod»:
   - бібліотеки та їхні версії;

- таблицю: що у вашій версії замінило `register`, `Controller`, `useWatch`, `handleSubmit`, `zodResolver` і `reset`;
- розмір JS-файлу збірки, який друкує `npm run build`, в обох версіях;
- які компоненти рендеряться під час введення в полі «Ім'я» в обох версіях (**Highlight updates when components render**);
- дату останнього релізу кожної бібліотеки (npmjs.com, вкладка **Versions** пакета);
- 3–5 речень: яку пару бібліотек ви взяли б для наступного проєкту й чому.

Останній реліз Formik, 2.4.9, вийшов у листопаді 2025 року. Formik використовувати можна, але цю дату запишіть у порівняння.

5. **Перевірки.** У `lab-6-alt/` `npm run lint` проходить без помилок і попереджень, `format:check` і `build` — без помилок. На Vercel імпортуйте ще один проєкт: Project Name `react-lab-6-alt`, Root Directory `lab-6-alt`.

Коміти: `chore: copy lab 6 for the alternative version`, потім `feat: …` з назвою зробленого (наприклад, `feat: rebuild the booking form with Formik and Yup`) і `docs: compare the form libraries`.

5. Вимоги до звіту

Тема листа:

```
[React] ЛР-6: складні форми та валідація
```

У листі вкажіть теку лабораторної на GitHub і адресу застосунку на Vercel:

```
GitHub repository: https://github.com/<username>/react-labs/tree/main/lab-6
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. знімок екрана форми після надсилання з порожніми полями: повідомлення під полями й курсор у полі «Ім'я»;
2. знімок екрана відповіді сервера на 19:00, коли значення лишилися в полях;
3. знімок екрана `npm run lint`, `npm run format:check` і `npm run build` без помилок;
4. чи виконано §4.7;
5. чи виконано §4.8: які бібліотеки ви вибрали й адреса `lab-6-alt` на Vercel (саме порівняння лежить у `lab-6-alt/README.md`);
6. чи використовували AI-агента і для чого;
7. короткий висновок: що зроблено й що було складним.

Звіт про основну частину можна надіслати до того, як ви закінчите §4.8. Тоді адресу `lab-6-alt` надішліть відповіддю на той самий лист до захисту.

6. Усний захист

Формат здачі передбачає усну співбесіду за результатами роботи. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного коду. Викладач може попросити показати відповідь у коді чи DevTools або змінити код наживо. Питання з груп «Форма без бібліотек» і «Інші інструменти» ставлять тим, хто виконав §4.7 і §4.8.

Питання для підготовки:

Валідація на клієнті й на сервері

- Навіщо перевіряти дані і на клієнті, і на сервері? Що станеться, якщо залишити лише одну з перевірок?
- Яке правило форми бронювання може перевірити лише сервер? Чому?
- Навіщо формі й серверу одна схема? Що піде не так, якщо записати правила двічі?
- Що станеться, якщо прибрати `resolver` з `useForm`? А якщо прибрати ще й `noValidate`?

Нативна валідація і бібліотеки

- Які правила цієї форми браузер міг би перевірити сам? Якими атрибутами?
- Що змінює `noValidate`? Навіщо полям лишили `required`?
- Що дали Zod і React Hook Form такого, чого не вміє перевірка браузера?
- Коли для форми досить можливостей браузера?

Zod

- Що таке схема? Що повертає `safeParse` і чим він відрізняється від `parse`?
- Чим `refine` відрізняється від `regex` чи `min`? Коли без `refine` не обійтися?
- Навіщо `z.coerce.number()`? Що отримала б схема без нього, якби дані прийшли з `FormData`?
- Як працює правило для двох полів? Навіщо `path: ['time']`?
- Що робить `trim()` і де це видно на сторінці?
- Скільки помилок у `issues` дає порожня дата і скільки повідомлень видно під полем? Чому?
- Чому повідомлення в схемі власні, а не стандартні повідомлення Zod?

React Hook Form

- Що повертає `register('name')`? Поле «Ім'я» контрольоване чи ні? Де зберігається його значення?
- Що рендериться, коли ви вводите ім'я? А коментар? Чому?
- Навіщо `Controller` для `GuestsStepper`? Чому `register` тут не підходить? Як це пов'язано з контрольованим компонентом із ЛР-5?
- Чим `useWatch` відрізняється від `watch`? Про що попереджає ESLint і чому?

- Що робить `handleSubmit(onValid)`? Чому тут функцію викликають у JSX, хоча в ЛР-5 так робити було не можна?
- Чим режими `onSubmit`, `onBlur`, `onChange` і `onTouched` відрізняються для людини, яка заповнює форму?
- Навіщо `defaultValues`? Що робить `reset()`?
- Чому `setFocus('name')` стоїть перед `reset()`? Що буде, якщо поміняти їх місцями?
- Навіщо `deps: 'time'` на полі дати?
- Чому «Очистити» тепер `type="button"` з `onClick`, а не `type="reset"` з `onReset` на формі?

Actions у React 19

- Що таке дія (Action)? Що повертає `useActionState`?
- Навіщо `startTransition` в `onValid`? Що сталося без нього?
- Навіщо другий `startTransition`, навколо `reset()` у `bookTable`?
- Що станеться, якщо натиснути «Забронювати» двічі? Як `useActionState` виконує кілька викликів і що робить перевірка `isPending`?
- Що станеться, якщо натиснути «Очистити», поки бронювання надсилається?
- Як скинути стан `useActionState`? Як це зроблено у вашій формі?
- Чому форма бронювання надсилається через `onSubmit`, а не через `<form action>`?
- Чому `bookTable` оголошено всередині компонента?

Доступність форми

- Що роблять `aria-invalid` і `aria-describedby`? Що почує користувач програми зчитування з екрана на полі з помилкою?
- Чому абзац із повідомленням стоїть поза `<label>`?
- Куди переходить фокус після надсилання з помилками і чому це важливо?
- Чому відповідь сервера — в `role="status"`, а повідомлення під полями — ні? Чим `role="status"` відрізняється від `role="alert"`?
- Чому кнопка під час надсилання має `aria-disabled`, а не `disabled`?
- Які поради з W3C WAI «Validating Input» і «User Notifications» ви застосували, а які — ні?

Форма без бібліотек

- Що отримує функція, передана в `action` форми? Хто її викликає?
- Навіщо `SubmitButton` окремий компонент? Що поверне `useFormStatus` у самому `SubscribeForm`?
- Що станеться, якщо натиснути кнопку ще раз, поки вона показує «Підписуємо...»? Якщо це проблема, як її виправити?
- Що відбувається з полем після дії? Як `defaultValue` повертає значення?
- Коли формі не потрібна бібліотека форм?

Інші інструменти

- Чому ви вибрали ці бібліотеки? Що замінило кожен API React Hook Form?
- Поля у вашій версії контрольовані чи ні? Що це означає для рендерів під час введення?
- Що довелося змінити в схемі? У чому нова бібліотека схем поводиться інакше, ніж Zod?
- Що показали розміри збірки?
- Чи варто брати бібліотеку, останній реліз якої вийшов рік тому? Як перевірити, чи її ще підтримують?

7. Критерії готовності

Основна частина (§4.1–4.6) оцінюється в 1 бал після захисту (§6). §4.7 і §4.8 — ще по 1 балу, теж після захисту. Це додаткові бали понад 65 балів за поточну роботу.

1 бал: §4.1–4.6

Критерій	Мінімальна вимога
Проєкт	`lab-6/` у репозиторії `react-labs`; у теці `npm run lint` проходить без помилок і попереджень, `format:check` і `build` — без помилок
Схема	усі правила форми — у `bookingSchema.js`, з повідомленнями українською; ту саму схему використовують форма й `api.js`
React Hook Form	поля через `register`, без `useState` для їхніх значень; `GuestsStepper` через `Controller`; лічильник через `useWatch`
Помилки	`noValidate`; повідомлення під полями з `aria-invalid` і `aria-describedby`; після порожнього надсилання курсор у першому неправильному полі; повідомлення про минулий час зникає після зміни дати
Надсилання	`useActionState` і `startTransition`; «Надсилаємо…» під час очікування; натискання під час «Надсилаємо…» не надсилає вдруге; після відмови значення лишаються, після успіху форма очищається, а підтвердження лишається
Очищення	«Очистити» очищає поля, повідомлення про помилки й відповідь сервера; курсор у полі «Ім'я»
GitHub	коміти з §4.6
На занятті	усі коміти §4.1–4.6 відправлено на занятті (час `git push` видно в **Activity** репозиторію)
Vercel	проєкт з Root Directory `lab-6`; адреса показує сторінку з останнього коміту `main`
Звіт	усі пункти §5

+1 бал: §4.7

Критерій	Мінімальна вимога
Форма	`SubscribeForm` з `<form action>`; дія отримує `FormData` і перевіряє адресу через `z.email()`

Критерій	Мінімальна вимога
Очікування	`SubmitButton` — окремий компонент із `useFormStatus`; «Підписуємо…» під час очікування
Значення	неправильна адреса лишається в полі після відповіді; після успішної підписки поле порожнє
На занятті	коміт §4.7 відправлено на занятті; Vercel показує останній коміт

+1 бал: §4.8

Критерій	Мінімальна вимога
Версія	`lab-6-alt/` у `react-labs` з власною назвою в `package.json`; React Hook Form замінено іншою бібліотекою форм; `npm run lint` без помилок і попереджень, `format:check` і `build` — без помилок
Поведінка	така сама, як у `lab-6` після §4.6: правила й повідомлення, ARIA, фокус на першій помилці, «Надсилаємо…», відповідь сервера, очищення після успіху, «Очистити»
Сервер	`api.js` перевіряє дані схемою цієї версії
Порівняння	розділ у `lab-6-alt/README.md`: таблиця відповідників, розміри збірки, рендери під час введення, дати останніх релізів, висновок
Vercel	проєкт з Root Directory `lab-6-alt`
Термін	коміти §4.8 відправлено до захисту ЛР-6